**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS**
**UNIDADE DE ENSINO DESCENTRALIZADA DE LEOPOLDINA**
**CURSO DE ENGENHARIA DE CONTROLE E AUTOMAÇÃO**

# AVALIAÇÃO 3



Disciplina: Programação de Computadores II (CMA04 / CMA05) — Turma: 2º Período
Semestre: 01/2014 — Data: **02/09/2014**
Professor: Anderson Grandi Pires — Valor: **25 pontos**

**Observações**
1. A avaliação é individual.
2. O aluno deverá selecionar uma única resposta em cada questão de múltipla escolha.

Aluno: _____

1) (2 pontos) Uma classe A está sendo implementada de modo que outras classes não sabem como as informações estão sendo armazenadas na classe A. Desse modo, alterações que forem necessárias na classe A não geram efeitos colaterais nas outras classes.

O conceito da Programação Orientada a Objetos que melhor descreve o parágrafo acima é:
a) acoplamento
b) coesão
c) design baseado em responsabilidades
d) encapsulamento

2) (2 pontos) Dado o programa em linguagem Java abaixo, assinale a alternativa que contém o conteúdo armazenado nas variáveis `cont` e `soma` na 9ª linha, além do número de vezes que o conteúdo da variável `soma` foi impresso na tela, nessa ordem.

```
1   public class Cont {
2     public static void main(String args[]) {
3       int soma = 0, cont = 1;
4       for(soma = cont; cont <= 4; cont++) {
5         soma += cont;
6         if(cont%2 == 0)
7           System.out.println(soma);
8       }
9       // Conteúdo de cont e soma nesta linha
10    }
11  }
12
```

a) 4, 7, 2 b) 4, 11, 2 c) 5, 11, 2 d) 5, 7, 1 e) 4, 11, 1

3) (2 pontos) Considerando as afirmações abaixo, julgue os itens:
I. A coesão descreve a interconexão das classes.
II. O bom design de classe exibe um grau elevado de coesão.
III. Em um projeto com um bom design, cada classe é amplamente independente e se comunica com outras classes por intermédio de uma pequena e bem definida interface.
IV. Em um bom projeto de classes busca-se um alto grau de acoplamento e uma baixa coesão.

Considerando as afirmações acima, julgue os itens:
a) apenas I e III estão corretas
b) apenas II e III estão corretas
c) apenas I, II e III estão corretas
d) apenas I e IV estão corretas
e) todas estão corretas

4) (3 pontos) Julgue cada um dos itens abaixo como verdadeiro (V) ou falso (F).

( ) A principal razão por trás do princípio de acoplamento é a reutilização. Outra razão é a legibilidade.

( ) Ao colocarmos os atributos de uma classe privados e oferecermos métodos de acesso e modificadores públicos estamos reduzindo o acoplamento dessa classe com outras classes do sistema.

( ) O encapsulamento adequado em classes reduz o acoplamento, porém não leva a um projeto aprimorado.

( ) A diretriz de encapsulamento visa ocultar informações de implementação da visualização, ou seja tal diretriz sugere que somente as informações sobre o que uma classe pode fazer devem ser visíveis externamente e não como ela é.

( ) O design baseado em responsabilidade expressa a ideia de que cada classe deve ser responsável por tratar seus próprios dados.

( ) A atividade de reestruturar um design existente para manter um bom design de classe quando a aplicação for modificada ou estendida é denominada refatoração.

---

O conjunto de classes abaixo deverá ser utilizado para desenvolver as próximas questões. As classes compõem um projeto de uma agenda telefônica. Nesta versão do projeto assume-se que não serão adicionadas na agenda duas pessoas que possuam o mesmo nome completo. Toda mudança deverá ser efetuada considerando os conceitos de design de projeto, ou seja, aplicando os conceitos de encapsulamento, coesão, acoplamento e design baseado em responsabilidades.

```
import java.util.HashMap;

public class AgendaTelefonica {
  private HashMap<String, Pessoa> contatos;

  public AgendaTelefonica() {
    contatos = new HashMap<String, Pessoa>();
  }

  public void add(Pessoa p) {
    contatos.put(p.nome + p.sobrenome, p);
  }
}
```

```
public class Pessoa {
  public String nome, sobrenome;
  public NumTelefone telefone;

  public Pessoa(String n, String s) {
    this.nome = n; this.sobrenome = s;
  }

  public void addTel(NumTelefone nt) {
    telefone = nt;
  }

  public String toString() {
    return nome + " " + sobrenome + ": " + telefone.getNum();
  }
}
```

```
public class NumTelefone {
  private String ddd, prefixo, numero;

  public NumTelefone(String d, String p, String n) {
    ddd = d; prefixo = p; numero = n;
  }

  public String getNum() {
    return ddd + prefixo + numero;
  }
}
```

```
public class Teste {
  public static void main(String args[]) {
    AgendaTelefonica agenda = new AgendaTelefonica();
    Pessoa p = new Pessoa("Joao", "Silva Lima");
    p.addTel(new NumTelefone("32", "3441", "0001"));
    agenda.add(p);
  }
}
```

5) (4 pontos) Algumas classes do projeto da agenda estão com acoplamento forte, além de apresentarem problemas no que diz respeito ao encapsulamento. Refatore as classes que julgar necessário de modo a melhorar o projeto da agenda. (Reescreva somente as classes que sofrerem mudança)

6) (2 pontos) Implemente um método na classe `NumTelefone` com assinatura `public String getNumFormat()` para retornar uma `String` que represente o número de telefone com a seguinte formatação: (`ddd`) `prefixo-numero`. (Exemplo: (32) 3449-2300)

7) (4 pontos) Modifique a classe `Pessoa` de modo que uma pessoa possa ter um número ilimitado de números de telefones. Reformule o método `toString()` para considerar essa modificação. (Inclua aqui somente as modificações)

8) (2 pontos) Modifique o corpo do método `main` da classe Teste de modo que a agenda possua dois contatos telefônicos, sendo que cada um deles tenha pelo menos 2 números de telefone.

9) (4 pontos) Apresente um diagrama de objetos para o cenário gerado à partir da execução do método `main` da questão anterior.